

Manual de Execução

Sistema de Controle Preditivo — Digital Twin do Processo Bayer

Versão: 1.0 · **Projeto:** projeto_bayer/ · **Data:** agosto/2026

Este manual descreve, passo a passo, como **instalar**, **configurar** e **executar** o ambiente de supervisão e controle do processo de produção de alumina (Processo Bayer), composto por um **Gêmeo Digital** multicapacidade, um **controlador Fuzzy Adaptativo** e um **agente orquestrado por LangGraph** com intervenção humana obrigatória (HITL).

1. Visão Geral

O sistema simula a etapa crítica de **controle de nível de decantadores** para evitar transbordamentos, especialmente sob chuva intensa. Ele integra:

- **Simulação física** dos tanques (série e paralelo) com balanço de volume e química.
- **Controlador Fuzzy Adaptativo** para abertura modulante das válvulas de drenagem.
- **Previsão meteorológica** em tempo real (OpenWeather).
- **Human-in-the-Loop (HITL)** — ações críticas só executam após aprovação do operador.
- **Persistência** em série temporal (InfluxDB) e **dashboard** interativo (Streamlit).

Arquitetura

Fluxo do agente: coleta → análise → calcular_controle → [crítico] aguardar_operador → executar_controle. Quando o nível (filtrado por EMA) supera o limite crítico sob chuva, o fluxo **interrompe** aguardando aprovação do operador antes de agir fisicamente nas válvulas.

Visão de sistema (versão implementada)

A descrição completa dos componentes, a comparação com o modelo conceitual e as métricas de validação estão em [docs/SISTEMA_IMPLEMENTADO.md](#).

2. Requisitos

- **Windows** (o ambiente foi desenvolvido e validado em Windows) — os comandos abaixo são para **PowerShell**.
- **Python 3.10 ou superior**.
- **Git** (opcional, para clonar/versionar).
- Opcionais por funcionalidade:

Funcionalidade	Dependência extra
Chuva real	Conta gratuita no OpenWeatherMap

Funcionalidade	Dependência extra
Persistência de série temporal Dashboard	InfluxDB 2.x (local ou via Docker) Pacotes streamlit, plotly, pandas (já em requirements.txt)

O **núcleo** (simulador + controladores + agente + testes) roda e é validado **sem** os serviços opcionais — eles degradam com segurança quando indisponíveis.

3. Instalação

3.1 Obter o código

```
cd C:\Users\lugos\Documents\Agente_Bayer
# (ou clone do repositório)
git clone https://github.com/LuisGSVasconcelos/bayer-digital-twin.git
cd bayer-digital-twin
```

3.2 Criar o ambiente virtual e instalar dependências

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Se preferir uv (mais rápido), o equivalente é:

```
uv venv .venv --seed
uv pip install --python .venv\Scripts\python.exe -r requirements.txt
```

4. Configuração (serviços opcionais)

Se quiser chuva real e persistência, crie o arquivo .env a partir do modelo:

Copy-Item .env.example .env

Preencha com sua chave e token:

```
# OpenWeatherMap
OPENWEATHER_API_KEY=sua_chave_aqui
OPENWEATHER_LAT=-23.5505
OPENWEATHER_LON=-46.6333
OPENWEATHER_UNITS=metric
OPENWEATHER_LANG=pt_br

# InfluxDB
```

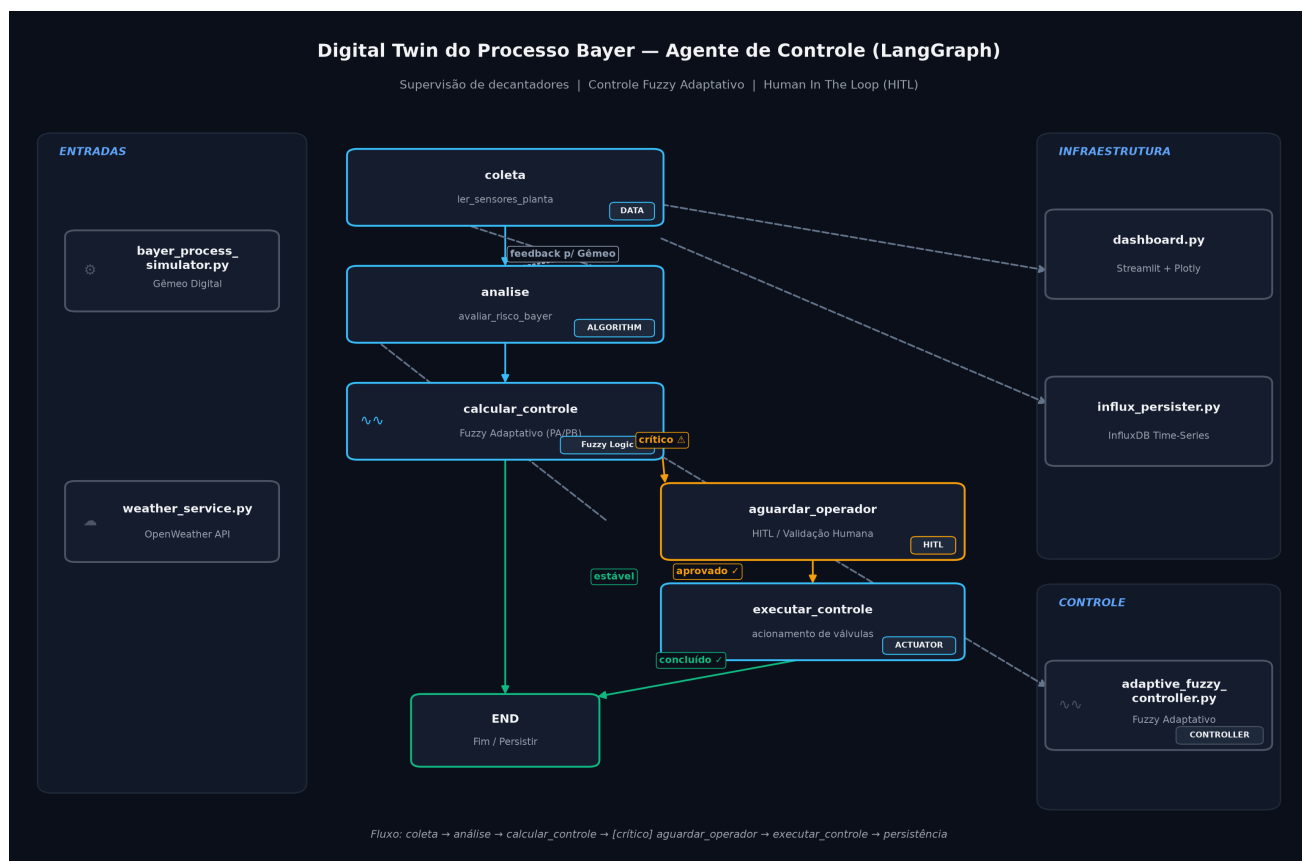


Figure 1: Arquitetura do Digital Twin

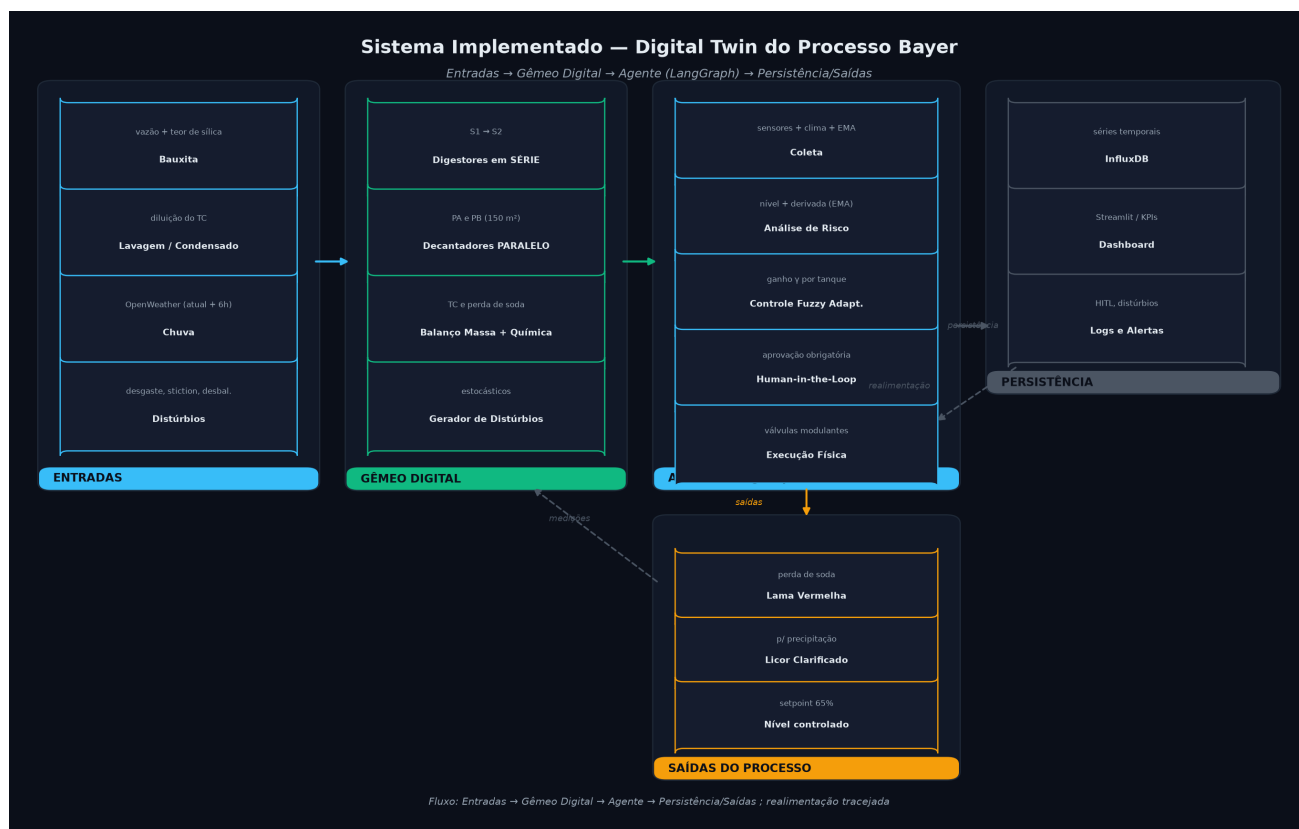


Figure 2: Sistema implementado

```
INFLUXDB_URL=http://localhost:8086
INFLUXDB_TOKEN=seu_token
INFLUXDB_ORG=minha_planta
INFLUXDB_BUCKET=processo_bayer
```

Sem esse arquivo ou sem as credenciais, o sistema roda normalmente usando clima padrão (“sem chuva”) e sem persistência — útil para desenvolvimento e testes offline.

5. Execução

Todas as etapas abaixo assumem o ambiente ativado (item 3.2).

5.1 Rodar os testes automatizados

```
pytest -v
```

Esperado: **23 testes aprovados** cobrindo controlador fuzzy, controlador adaptativo, simulador físico e fluxo do agente (HITL), incluindo a correção do domínio do erro.

5.2 Benchmark comparativo (Padrão vs Adaptativo)

Cria métrica **reproduzível** (seeds fixas, 10 repetições, média $\pm$ desvio-padrão):

```
python test_adaptive_controller.py
```

Saída esperada (valores de referência):

Fuzzy Padrão: RMSE 8.14% $\pm$ 0.00

Fuzzy Adaptativo: RMSE 7.81% $\pm$ 0.00

Melhoria: +4.1%

5.3 Executar o agente em modo texto (terminal)

Executa ciclos de supervisão e, quando detecta risco crítico, **pausa pedindo aprovação**:

```
python langgraph_agent.py
```

Em caso de alerta, o console pergunta Autorizar abertura emergencial das válvulas? (s/n). Digite s para liberar a ação ou n para abortar.

5.4 Dashboard interativo (Streamlit)

```
streamlit run dashboard.py
```

Abra no navegador: **http://localhost:8501**

No painel: - clique **Iniciar** para rodar a simulação e **Parar** para pausar; - ajuste a **velocidade** (ciclos/s); - quando o alerta de HITL surgir, clique **Aprovar Ação Emergencial** para

liberar a válvula; - acompanhe KPIs (níveis, TC, perda de soda) e gráficos (PV × SP × MV, química, distúrbios).

6. Estrutura de Arquivos

```
projeto_bayer/
├── bayer_process_simulator.py # Gêmeo Digital (tanques, decantadores, distúrbios)
├── fuzzy_controller.py        # Controlador Fuzzy base (Mamdani + centroide)
├── adaptive_fuzzy_controller.py # Fuzzy com ganho adaptativo (gradiente + momentum)
├── weather_service.py         # Integração OpenWeather (opcional)
├── langgraph_agent.py         # Agente LangGraph + HITL
├── influx_persister.py        # Persistência InfluxDB (opcional)
├── dashboard.py               # Dashboard Streamlit + Plotly
├── test_adaptive_controller.py # Benchmark reproduzível
├── tests/                      # Suíte pytest (23 testes)
├── docs/
│   └── SISTEMA_IMPLEMENTADO.md # Visão completa do sistema (componentes, métricas)
├── scripts/
│   ├── diagrama_arquitetura.py # Gera o diagrama do fluxo (PNG dark-mode)
│   └── diagrama_sistema_implementado.py # Gera o diagrama em subgrafos (PNG dark-mode)
├── assets/
│   ├── arquitetura_processo_bayer.png
│   └── arquitetura_sistema_implementado.png
├── requirements.txt
├── .env.example                # Modelo de configuração de serviços
└── MANUAL_EXECUCAO.md         # Este documento
```

7. Solução de Problemas (Troubleshooting)

Sintoma	Provável causa	Solução
pytest não encontrado	Ambiente não ativado / deps ausentes	Ative o venv e rode <code>pip install -r requirements.txt</code>
Erro ao importar <code>influxdb_client</code>	Pacote não instalado	<code>pip install influxdb-client</code> (opcional; sem ele a persistência vira no-op)
Clima sempre “sem chuva” / “indisponível”	Sem chave OpenWeather ou sem <code>.env</code>	Preencha <code>OPENWEATHER_API_KEY</code> no <code>.env</code>
Dashboard não inicia	<code>streamlit/plotly</code> ausentes, ou porta em uso	<code>pip install streamlit plotly pandas</code> ; troque a porta com <code>--server.port 8502</code>

Sintoma	Provável causa	Solução
Branch master vs main	Inicialização Git	Envie com <code>git push -u origin main</code>
Autenticação GitHub pedindo senha	Token ausente	Usar Windows Credential Manager ou token classic com escopo repo

Regenerar o diagrama de arquitetura

```
python scripts\diagrama_arquitetura.py
```

8. Referência Rápida de Comandos

```
# instalação
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt

# configuração (opcional)
Copy-Item .env.example .env

# validação
pytest -v
python test_adaptive_controller.py

# aplicação
streamlit run dashboard.py           # dashboard web
python langgraph_agent.py           # agente em modo texto (HITL interativo)
```

Documento gerado como artefato do projeto projeto_bayer — Processo Bayer (alumina).